

移动机器人规划与控制实验报告

23354090 李炬熹

1 题目一

1.1 问题描述

已知末端执行器固连坐标系 $\{D\}$ 相对于机体坐标系 $\{B\}$ 的姿态变化为：

$${}^B R_D = \begin{bmatrix} \cos \omega t & -\sin \omega t \cos \alpha & \sin \omega t \sin \alpha \\ \sin \omega t & \cos \omega t \cos \alpha & -\cos \omega t \sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix} \quad (1)$$

其中， $\omega = 0.5 \text{ rad/s}$ ， $\alpha = \pi/12$ 。

要求：计算执行器在世界坐标系 (World Frame) 下的姿态 (以四元数表示)，绘制出四元数的变化曲线。为保证四元数时间序列的连续性，需确保四元数归一化，且 $q_w \geq 0$ 。

1.2 计算思路

Step 1: 获取机体姿态

从轨迹数据 `tracking.csv` 中读取每个时刻 t 的机体姿态四元数 $\mathbf{q}_{body}$ ，该四元数表示从世界坐标系到机体坐标系的旋转。将其转换为旋转矩阵 R_{bw} (world $\rightarrow$ body)，再求转置得到 R_{wb} (body $\rightarrow$ world)。

Step 2: 构造末端执行器相对旋转矩阵

根据给定公式，计算末端执行器相对于机体的旋转矩阵 R_{bd} (body $\rightarrow$ device)：

$$R_{bd} = \begin{bmatrix} \cos(\omega t) & -\sin(\omega t) \cos \alpha & \sin(\omega t) \sin \alpha \\ \sin(\omega t) & \cos(\omega t) \cos \alpha & -\cos(\omega t) \sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix} \quad (2)$$

Step 3: 计算世界坐标系下的姿态

通过矩阵乘法组合两个旋转，得到末端执行器在世界坐标系下的姿态：

$$R_{wd} = R_{wb} \cdot R_{bd} \quad (3)$$

Step 4: 旋转矩阵转四元数

将旋转矩阵 R_{wd} 转换为四元数 $\mathbf{q} = (q_x, q_y, q_z, q_w)$ ，并进行归一化处理。为保证时间序列的连续性，若 $q_w < 0$ 则将整个四元数取反。

1.3 实验结果

按轨迹数据 ($t \in [0, 30]$ s) 采样, 计算各时刻的末端执行器姿态四元数, 结果保存至 task1_quaternion.csv。

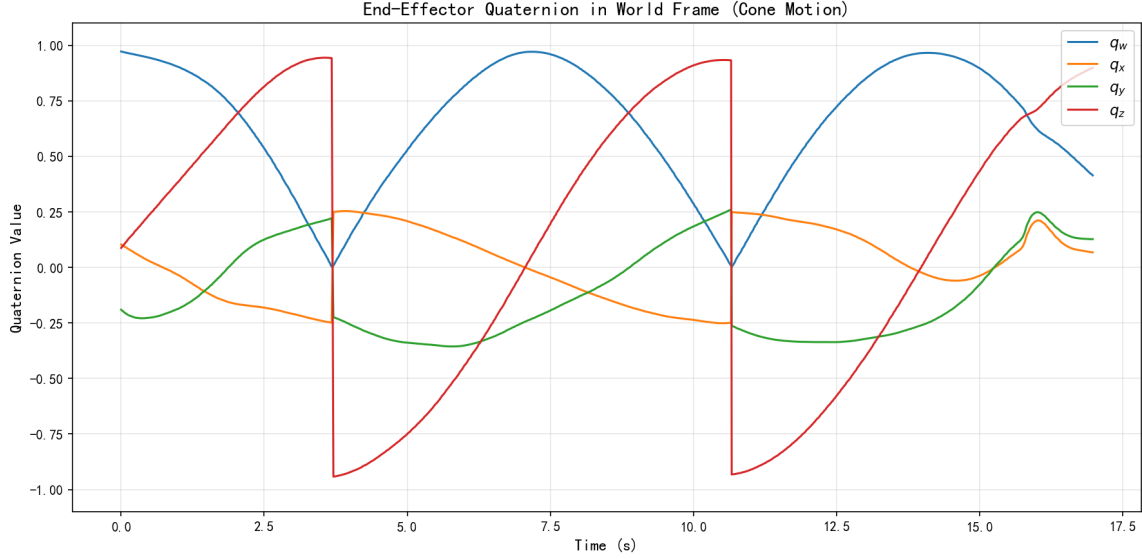


图 1: 末端执行器姿态四元数随时间变化曲线

从图中可以观察到:

- 四元数各分量呈现周期性变化, 周期约为 $T = 2\pi/\omega \approx 12.57$ s
- q_w 分量始终保持非负, 满足连续性约束
- 曲线平滑连续, 没有跳变, 验证了归一化和符号处理的正确性

2 题目二

2.1 问题 1: A* 轨迹规划的启发式与路径简化

tie_breaker 的作用: tie_breaker 通常通过对启发式值作极小幅度放大 (例如 $h \leftarrow 1.0001h$), 在 $f = g + h$ 相同的候选节点之间打破对称性。其效果是让搜索更倾向沿近似直线方向推进, 减少“之”字形扩展, 进而提升搜索效率并改善路径平滑性。

偏好平面飞行的修改方案: 若希望规划结果尽量“平飞”, 可以在 z 方向引入更高权重或代价, 使算法在可行时优先选择水平移动。一个直接做法是修改启发式度量为

$$h = \sqrt{dx^2 + dy^2 + w_z dz^2}, \quad w_z > 1,$$

或在边代价中对垂直运动加罚:

$$cost = \sqrt{dx^2 + dy^2} + \alpha |dz|, \quad \alpha > 1.$$

这样会显著抑制频繁爬升/下降，但代价是遇到障碍或必须换层时，路径可能更长、机动更保守。

path_resolution 对跟踪的影响： path_resolution 过大意味着航点保留得过密，轨迹会呈现更多细碎转折，跟踪器需要频繁调整姿态与速度，飞行时间可能增加，但路径更贴合原始 A* 输出；反之若过小则会把局部转折“抹平”为大跨度直线段，虽然看起来更简洁，但可能擦过障碍物边缘带来碰撞风险，同时在动力学约束（最大加速度/角速度）下更容易出现跟踪误差或拐弯跟不上。

2.2 问题 2：SO(3) 位置控制器的力姿态生成

各项作用： $k_x e_x$ 负责把位置误差转化为“恢复力”，把系统拉回期望位置； $k_v e_v$ 相当于阻尼项，用于抑制振荡与超调； a_{des} 是前馈项，用于提前补偿期望加速度、提高跟踪响应； $g e_3$ 用于重力补偿，避免控制器把推力浪费在对抗重力上； $k_a a_{imu}$ 则引入基于 IMU 的加速度反馈，能够在扰动或快速机动时提升响应速度与鲁棒性。

k_a 截断原因： 当误差较大或 IMU 瞬时噪声较强时， $k_a a_{imu}$ 可能被放大到不合理的量级，造成推力/力矩指令饱和并引入高频抖动，表现为电机输出突变甚至闭环不稳定。对该项进行限幅/截断可以避免控制量尖峰，提升系统安全性与稳定性。

重机体参数调整： 若机体质量 m 增大，为保持相近的闭环动态（例如相近的响应速度与阻尼比），通常需要相应提高位置与速度反馈增益，可近似按比例缩放，如 $k'_x \approx k_x \cdot (m'/m)$ ， k_v 同理。与此同时，所需总推力上升，姿态倾角更容易接近限幅；为了避免长期贴近饱和，需适当降低期望加速度（特别是横向加速度）或在规划/控制中收紧机动指令。

2.3 问题 3：动力学建模与约束

建模简化的影响： 若忽略气动阻尼，高速段的速度衰减特性会与真实系统不一致，导致跟踪出现相位偏差（例如减速段提前或滞后）；质量或惯量建模误差会使推力与力矩分配不准，表现为姿态响应偏慢、过冲增大或跟踪精度下降。

参数估计误差的可观测现象： 质量估计偏大通常表现为“推不动”：给定同样的控制指令时加速度达不到期望，出现稳态下沉或爬升乏力；阻尼估计偏小会导致高速段减速不足、过冲与回摆更明显；惯量误差更常体现在姿态环上，如角速度跟踪不准、姿态振荡或某一轴响应明显偏慢/偏快。

气动阻力 $F_d = -k_v v$ 的加入位置： 更合理的做法是以控制层补偿为主、规划层留安全裕量：在 SO3Control 中显式加入 F_d 的补偿可以获得更快的实时响应，但前提是速度测量可靠且 k_v 估计不过分失真；在规划层通过降低最大速度/加速度约束则更保守，能提升安全性但会牺牲性能。实际调参可从较小的 k_v 起步，逐步增大并观察高速段跟踪误差与振荡趋势，直至在稳定性与精度之间达到折中。

3 题目三

3.1 任务描述

根据给定的双纽线轨迹，利用微分平坦性推导无人机在飞行过程中的姿态四元数。偏航角 ψ 与速度方向对齐。

轨迹方程（世界坐标系， $t \in [0, 2\pi]$ ）：

$$x = \frac{10 \cos t}{1 + \sin^2 t}, \quad y = \frac{10 \sin t \cos t}{1 + \sin^2 t}, \quad z = 10 \quad (4)$$

3.2 推导思路

四旋翼微分平坦性的核心：推力方向（机体 z 轴）由期望加速度决定。

Step 1： 计算机体 z 轴方向

$$\mathbf{z}_B = \frac{\mathbf{a} + g\mathbf{e}_3}{\|\mathbf{a} + g\mathbf{e}_3\|} \quad (5)$$

Step 2： 计算偏航角（与速度对齐）

$$\psi = \arctan 2(v_y, v_x) \quad (6)$$

Step 3： 构造机体坐标系

$$\mathbf{y}_B = \frac{\mathbf{z}_B \times \mathbf{x}_C}{\|\mathbf{z}_B \times \mathbf{x}_C\|}, \quad \mathbf{x}_B = \mathbf{y}_B \times \mathbf{z}_B \quad (7)$$

其中 $\mathbf{x}_C = (\cos \psi, \sin \psi, 0)^T$ 。

Step 4： 旋转矩阵转四元数

$$R = [\mathbf{x}_B, \mathbf{y}_B, \mathbf{z}_B] \rightarrow \mathbf{q} = (q_w, q_x, q_y, q_z) \quad (8)$$

3.3 实验结果

按步长 $\Delta t = 0.02\text{s}$ 采样，计算 316 个时刻的姿态四元数，结果保存至 `df_quaternion.csv`。

3.4 运行命令

```
1 cd MRPC-2025-homework/code
2 source devel/setup.bash
3 rosrn quadrotor_df df_quaternion_node
```

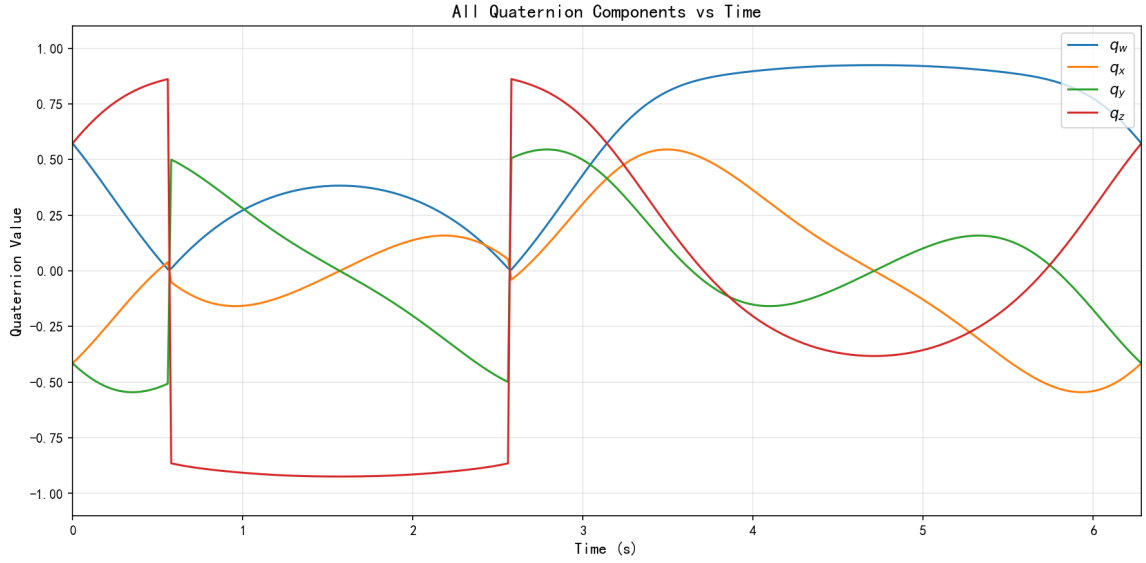


图 2: 四元数分量随时间变化曲线

4 任务一：四旋翼飞机动力学模型补全

4.1 任务描述

在 `Quadrotor.cpp` 中补充四旋翼飞机的动力学模型，包括速度导数 $\dot{v}$ 和角速度导数 $\dot{\omega}$ 。

4.2 理论分析

4.2.1 速度导数 $\dot{v}$

根据牛顿第二定律，四旋翼在世界坐标系下的平动方程为：

$$m\dot{v} = F_{gravity} + F_{thrust} + F_{external} + F_{drag} \quad (9)$$

各力分量：

- **重力：** $F_{gravity} = m \cdot (0, 0, -g)^T$
- **推力：** 推力沿机体 z 轴方向，需通过旋转矩阵 R 转换到世界坐标系

$$F_{thrust} = R \cdot \begin{pmatrix} 0 \\ 0 \\ T \end{pmatrix} = T \cdot R_{:,3} \quad (10)$$

- **空气阻力：** 与速度方向相反， $F_{drag} = -C_{drag} \cdot \|v\|^2 \cdot \hat{v}$
- **外力：** $F_{external}$ (如风扰动)

因此：

$$\dot{v} = \begin{pmatrix} 0 \\ 0 \\ -g \end{pmatrix} + \frac{T}{m} R_{:,3} + \frac{F_{external}}{m} - \frac{resistance}{m} \hat{v} \quad (11)$$

4.2.2 角速度导数 $\dot{\omega}$

根据刚体旋转的欧拉方程：

$$J\dot{\omega} + \omega \times (J\omega) = \tau + \tau_{external} \quad (12)$$

其中 J 为惯性矩阵， τ 为电机产生的力矩， $\omega \times (J\omega)$ 为科里奥利力项。

求解得：

$$\dot{\omega} = J^{-1}(\tau - \omega \times (J\omega) + \tau_{external}) \quad (13)$$

4.3 实现代码

```
1 // 速度导数
2 Eigen::Vector3d gravity(0, 0, -g_);
3 Eigen::Vector3d thrust_force = R.col(2) * thrust / mass_;
4 Eigen::Vector3d drag_force = -resistance * vnorm / mass_;
5 Eigen::Vector3d external_force = external_force_ / mass_;
6 v_dot = gravity + thrust_force + external_force + drag_force;
7
8 // 角速度导数（欧拉方程）
9 omega_dot = J_.inverse() * (moments
10     - cur_state.omega.cross(J_ * cur_state.omega)
11     + external_moment_);
```

4.4 实验结果

编译通过后，运行悬停测试：

```
roslaunch trajectory_generator test_control.launch
```

无人机能够稳定悬停在约 3 米高度，验证了动力学模型的正确性。

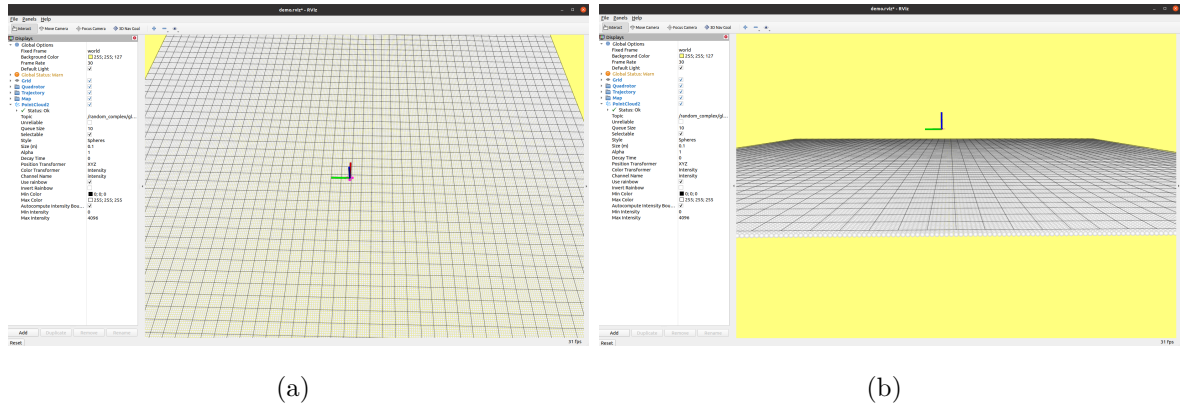


图 3: 任务一结果

5 任务二：A* 路径规划算法补全

5.1 任务描述

在 `Astar_searcher.cpp` 中补充 A* 算法的启发式函数、主循环和路径追溯。

5.2 算法原理

A* 算法通过评估函数 $f(n) = g(n) + h(n)$ 选择最优节点，其中 $g(n)$ 为实际代价， $h(n)$ 为启发式估计。

5.3 补全思路

STEP 1.1 启发式函数：采用欧几里得距离并添加 `tie_breaker` 加速收敛：

```
1 // p 值的选择通常设为：1.0 / 地图尺寸，相比不添加有微小的优化
2 heu = sqrt(dx*dx + dy*dy + dz*dz) * (1 + 1/2500);
```

STEP 1.2 主循环：实现标准 A* 流程——从 Open Set 取 f 值最小节点，检查终点，扩展 26 邻域邻居并更新代价：

```
1 currentPtr = Openset.begin()->second; // 取最优节点
2 if (currentPtr->index == goalIdx) return true; // 终点检查
3 // 更新邻居
4 neighborPtr->g_score = tentative_g_score;
5 neighborPtr->f_score = g_score + getHeu(neighborPtr, endPtr);
6 neighborPtr->Father = currentPtr; // 记录父节点
```

STEP 1.3 路径追溯：沿 Father 指针回溯并反转：

```
1 while(terminatePtr != NULL) {
2     front_path.push_back(terminatePtr);
3     terminatePtr = terminatePtr->Father;
4 }
```

关键点：必须使用 Map_Node 中的预分配节点，否则 Father 指针无法回溯。

5.4 实验结果

- 1 计算得到的均方根误差 (RMSE) 值为：0.6106656332100661
- 2 轨迹运行总时间为：78.01016068458557
- 3 总轨迹长度为：65.11170127984401
- 4 是否发生了碰撞：0
- 5 综合评价得分为(综合分数越低越好)：150.75749903489913

运行 demo 后，无人机成功规划路径并飞行至目标位置。

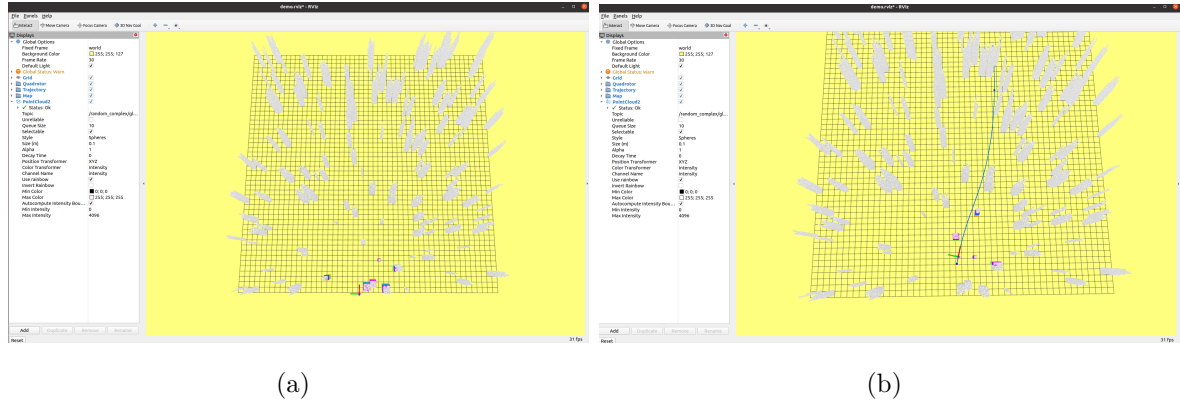


图 4: 任务二实验结果

6 任务三 + 附加题：算法优化与性能改进

6.1 优化目标

在基础 A* 算法和控制器实现的基础上，通过多方面优化降低综合评价得分。评分公式为：

$$Score = 200 \times RMSE + 0.2 \times T_{total} + 0.2 \times L_{total} + 40 \times Collision \quad (14)$$

6.2 优化方法

6.2.1 启发式函数改进

将欧几里得距离改为 **3D 对角线距离**，更适合 26 邻域的栅格移动：

$$h(n) = 0.318 \cdot d_{min} + 0.414 \cdot d_{mid} + d_{max} \quad (15)$$

其中 $d_{min}, d_{mid}, d_{max}$ 分别为三轴距离差的最小、中间和最大值。


```

1 double min_delta = std::min({dx, dy, dz});
2 double max_delta = std::max({dx, dy, dz});
3 double mid_delta = dx + dy + dz - min_delta - max_delta;
4 heu = 0.317837*min_delta + 0.414213*mid_delta + max_delta;

```

6.2.2 路径简化优化

将 Douglas-Peucker 算法替换为基于**视线检测 (LOS)** 的贪心简化:

```

1 // 从当前点寻找最远可直达点
2 for (size_t i = path.size()-1; i > current+1; i--) {
3     if (lineOfSight(path[current], path[i])) {
4         farthest = i; break;
5     }
6 }

```

该方法在保证路径安全的前提下, 最大程度减少航点数量

上面这两部分使得路径的选择有了很大的提升, 一开始用欧几里得距离的时候无人机经常出现重新飞回起点再起飞的现象, 而且在靠近障碍物时会出现十几个拐点聚在一起的现象, 大大降低了运行的速度以及运行的精度。在改进启发式距离后分数从原有的上百分下降到几十分。

6.2.3 时间分配优化

将轨迹时间裕量系数从 2.0 降低至 1.5, 使轨迹执行速度提升约 25%:

```

1 return 1.5 * time; // 原为 2.0 * time

```

这部分使得模型的运行轨迹更为清晰, 并减少了转弯的次数, 大大减少了运行速度。

6.2.4 控制器增益调整

增大位置和速度反馈增益, 提高跟踪精度:

```

1 kx_ = Eigen::Vector3d(25.0, 25.0, 16.0); // 原(15.0, 15.0, 12.0)
2 kv_ = Eigen::Vector3d(10.0, 10.0, 8.0); // 原(7.5, 7.5, 6.5)

```

6.3 调参思路与技巧

6.3.1 控制器增益调参

控制器采用 PD 结构: $F = k_x(x_{des} - x) + k_v(\dot{x}_{des} - \dot{x})$ 。调参遵循以下原则:

- **阻尼比匹配:** 保持 $k_v \approx 2\sqrt{k_x}$ 以接近临界阻尼, 避免振荡
- **渐进调整:** 先增大 k_x 提高响应速度, 再调整 k_v 抑制超调

这部分在调参的时候发现 k_x 越大, RMSE 相对越小, 但太大会导致无人机震荡 (例如 (40.0, 40.0, 25.0)), 于是选择当前参数作为最终参数, 能够保持 RMSE 较小的同时不会使无人机震荡。

6.3.2 时间分配调参

时间裕量系数的选择需权衡速度与跟踪精度:

- 系数过大 (如 2.0): 轨迹执行慢, 耗时长, 而且 RMSE 也偏高
- 系数过小 (如 1.2): 轨迹过快, 控制器跟踪困难, RMSE 升高, 并且容易出现路径错误的情况
- 最终选择 1.53 作为折中, 兼顾速度与精度

6.3.3 调参流程

采用迭代优化策略: 每次仅修改单一参数, 观察 RMSE、时间等指标变化, 多次实验取最优参数组合。

6.4 实验结果

```
1 计算得到的均方根误差 (RMSE) 值为: 0.022071263244566982
2 轨迹运行总时间为: 23.51840591430664
3 总轨迹长度为: 28.646625139333747
4 是否发生了碰撞: 0
5 综合评价得分为(综合分数越低越好): 14.847258859641475
```

6.5 对比

表 1: 优化前后性能对比

指标	优化前	优化后
RMSE	0.610	0.022
运行时间 (s)	78.011	23.518
轨迹长度 (m)	65.111	28.647
碰撞	0	0
综合得分	150.757	14.847

这里的综合得分因为随机数或者底层计算的原因会在 14-18 波动, 这里保存的是最优的得分。

7 总结

本次实验完成了移动机器人规划与控制的全部任务，主要工作如下：

理论分析部分

- **题目一：**完成末端执行器姿态四元数计算，通过组合无人机本体姿态与末端执行器圆锥运动，实现了世界坐标系下的姿态表示
- **题目二：**深入分析了 A* 算法的启发式设计、SO(3) 控制器各项作用、以及动力学建模对控制性能的影响
- **题目三：**基于微分平坦性理论，推导了双纽线轨迹下无人机的姿态四元数计算方法

代码实现部分

- **任务一：**实现四旋翼动力学模型，包含重力、推力、阻力和欧拉旋转方程
- **任务二：**完成 A* 算法实现，包括启发式函数、主循环和路径追溯
- **任务三 + 附加题：**通过启发式函数改进（3D 对角线距离）、LOS 路径简化、时间分配优化（1.53 倍裕量）和控制器调参，将综合得分从 150.757 降至 **14.847**